

Algorithmique Avancée**Examen Réparti 1**

Les seuls documents autorisés sont les polys de cours, ainsi que la copie personnelle. Le barème donné est indicatif.

Exercice 1 : QCM [4 points]

Dans ce QCM, pour chaque question vous devez donner 1 seule réponse et expliquer votre choix par une ligne de texte ou une figure. Le barème est le suivant : **réponse correcte et correctement argumentée : 1 point. Réponse incorrecte ou correcte mais mal argumentée : 0 point.** *Il n'y a pas de points négatifs.*

Question 1 Pour un arbre bicolore contenant n clés, l'algorithme permettant de calculer le plus petit élément a une complexité au pire (en nombre de nœuds traversés) en

A. $\Theta(1)$. B. $\Theta(\log n)$. C. $\Theta(n)$.

Décrire succinctement cet algorithme.

Question 2 Laquelle de ces 3 suites d'inégalités d'ordres de grandeur est correcte, c'est-à-dire correcte à partir d'un certain rang $n \in \mathbb{N}$ (expliquer pourquoi les autres sont incorrectes)

A. $3^n < n^2 2^n < n! < n^{10n}$.

B. $n^4 \log n < n^5 < n^4 2^n$.

C. $2^n < n! < (\sqrt{n})^n$.

Question 3 En utilisant un splay-tree de n clés, on peut réaliser un tri des n clés

A. en temps amorti $O(n \log n)$.

B. en temps moyen $O(n \log n)$.

C. en temps pire $O(n \log n)$.

Question 4 Le nombre de comparaisons nécessaires pour faire la fusion d'une file binomiale de 1221 éléments avec une file binomiale de 829 éléments est

A. 15.

B. 12.

C. 10.

Exercice 2 : Exercices [6 points]

Question 1 Donner un algorithme en $O(n \log n)$ comparaisons, qui étant donné une suite S de n nombres entiers et un nombre donné x , détermine s'il existe deux nombres p_1 et p_2 dans S tels que $p_1 + p_2 = x$.

Question 2 Soit un arbre 2-3-4 qui contient n clés. Supposons que son sous-arbre gauche ne contienne que des 2-nœuds et son sous-arbre droit que des 4-nœuds. Montrer que le sous-arbre gauche contient $\Theta(\sqrt{n})$ nœuds.

Question 3 On veut incrémenter un compteur binaire représenté par un tableau $A[0..k-1]$ de k bits (le tableau contenant le bit b_i en case i représente l'entier $\sum b_i 2^i$).

1. Écrire l'algorithme pour passer de x à $x + 1$ dans le tableau (pour $0 \leq x \leq 2^k - 2$). Quel est le nombre maximum de flips (passage d'un bit de 1 à 0 ou de 0 à 1) pour une opération d'incrément ?
2. On montre ici que le coût amorti, en nombre de flips, pour incrémenter le compteur de 0 à $n = 2^k - 1$ est de $O(1)$ pour une opération d'incrément.
 - (a) Méthode par agrégat : montrer que le nombre total de flips pour la suite des n incréments est majoré par $2n$.
 - (b) Méthode de potentiel : on considère la fonction qui à un tableau associe son nombre de bits à 1. Montrer que c'est une fonction de potentiel. Quel est le coût amorti de l'opération d'incrément de i vers $i + 1$?

Exercice 3 : Concaténation d'arbres 2-3-4 [14 points]

On considère le modèle des arbres 2-3-4 avec éclatements à la descente.

Question 1 Construire l'arbre 2-3-4, noté Ex_1 , obtenu en insérant les clés dans l'ordre suivant : 5, 7, 13, 1, 18, 17, 9, 4, 3, 12, 15. On indiquera toutes les étapes intermédiaires.

Question 2 Construire l'arbre 2-3-4, noté Ex_2 , obtenu en insérant les clés 25, 20, 22, 28, 21, 30, 26. On peut directement donner l'arbre final sans les étapes intermédiaires.

Question 3 Donner un algorithme (son pseudo-code ou sa description détaillée) construisant la liste, dans l'ordre croissant, des clés d'un arbre 2-3-4.

Question 4 Donner le pseudo-code d'un algorithme prenant en entrée un arbre 2-3-4, noté A , et renvoyant un couple composé de la clé de valeur maximale de A et de l'arbre issu de A en supprimant cette clé.

La suite de l'exercice consiste à obtenir une méthode efficace pour concaténer deux arbres 2-3-4, notés par la suite A_1 et A_2 tels que les clés de A_1 sont toutes plus petites (strictement) que les clés de A_2 .

Question 5 On propose une première méthode naïve. On insère les clés de A_2 successivement dans A_1 . Donner le pseudo-code d'un algorithme opérant selon ce principe. On utilisera la Question 3.

Question 6 Appliquer la méthode précédente aux arbres Ex_1 issu de la Question 1 et Ex_2 issu de la Question 2.

Question 7 Dans le cas général, en supposant que les deux arbres A_1 et A_2 sont de taille similaire n , quelle est la complexité de cette méthode en nombre de comparaisons ?

Nous allons désormais proposer une méthode plus efficace. On suppose que la hauteur h_1 de A_1 est supérieure ou égale à h_2 , celle de A_2 .

Voilà les idées globales de la méthode

- On descend sur la branche droite de A_1 jusqu'au nœud x de profondeur $h_1 - h_2$. Soit A_x l'arbre enraciné en x .
- On extrait la clé c , de valeur maximale de A_x (et on obtient donc $\tilde{A}_x$).
- On crée un nœud contenant les clés de x , suivies de c , suivies des clés de la racine de A_2 . On donne pour fils à ce nouveau nœud, les fils de la racine de $\tilde{A}_x$ suivis des fils de la racine de A_2 .
- Alors, soit le nouveau nœud vérifie la contrainte de taille des arbres 2-3-4, et alors on a fini après remplacement de A_x par ce nouvel arbre dans A_1 .
- Soit il faut faire des éclatements à la remontée.

Question 8 Décrire avec des schémas les différents cas d'éclatements pouvant se présenter, et comment chacun d'eux est traité.

Question 9 Donner le pseudo-code de cet algorithme.

Question 10 Quelle est la complexité de cet algorithme, en nombre de comparaisons ?